

# СОФТУЕРНИ АРХИТЕКТУРИ

RESTful Web Services



# КАКВО Е REST?

- **Representational State Transfer**
- Дефиниран през 2000 г. от Roy Fielding от Калифорнийския университет в дисертацията му на тема „[Architectural Styles and the Design of Network-based Software Architectures](#)“
- Stateless, client-server, cacheable, communication protocols
- Използване на HTTP заявки за запис, четене и изтриване на данни
- Лек
- Съществителните като URI, глаголите като HTTP методи

# КАКВО НЕ Е REST?

- Не е стандарт
- Не е протокол
- Не е заместител на SOAP

Какво е тогава?

Архитектурен шаблон

# РЕСУРСИ И КОНТЕЙНЕРИ

- Ресурс
  - Всичко, което е достатъчно важно за да бъде референцирано като отделна единица
  - Може да бъде физически обект или абстрактна концепция
  - Всеки ресурс има поне едно име
- Ресурсите могат да бъдат контейнери за други ресурси
  - Контейнерите винаги са ресурси

# ПРЕДСТАВЯНЕ НА РЕСУРСИ

- Представяне на ресурс
  - Всяка полезна информация за състоянието на ресурса
  - Специфицира се чрез URI или съдържание на хедърите

# ПРИМЕРНИ URI

- GET  
<http://www.example.com/v1/books/3245>
- POST  
<http://www.example.com/v1/books>
- DELETE  
<http://www.example.com/v1/books/3245>
- PUT  
<http://www.example.com/v1/books/3245>

# АРХИТЕКТУРА ОРИЕНТИРАНА КЪМ РЕСУРСИ

- Адресируемост
  - Ресурсите се идентифицират чрез уникални URI
  - Трябва да са описателни
  - Всеки URI идентифицира точно един ресурс

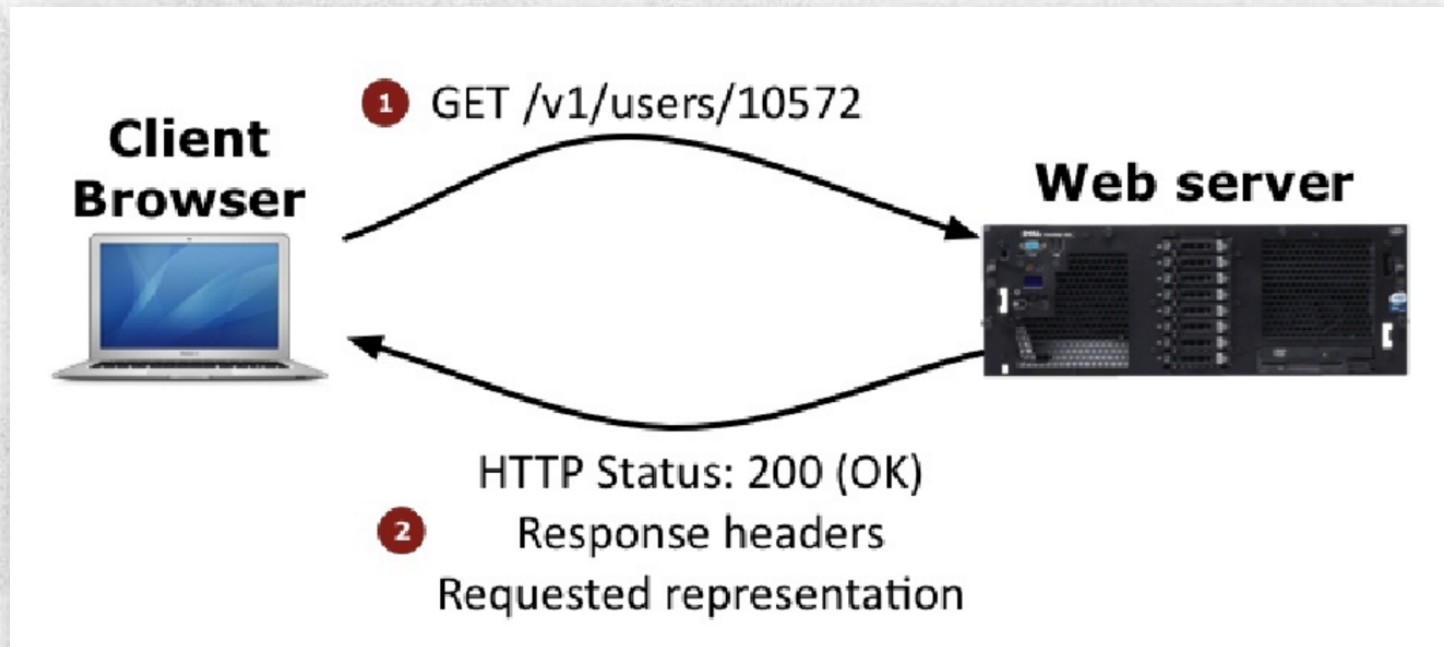
# АРХИТЕКТУРА ОРИЕНТИРАНА КЪМ РЕСУРСИ

- Липса на състояние
  - Не се поддържа състояние на връзката между отделните REST извиквания
  - Всяка HTTP заявка се изпълнява в пълна изолация на сървъра
  - Да се избягват използването на сесия и cookies
  - Без странични ефекти

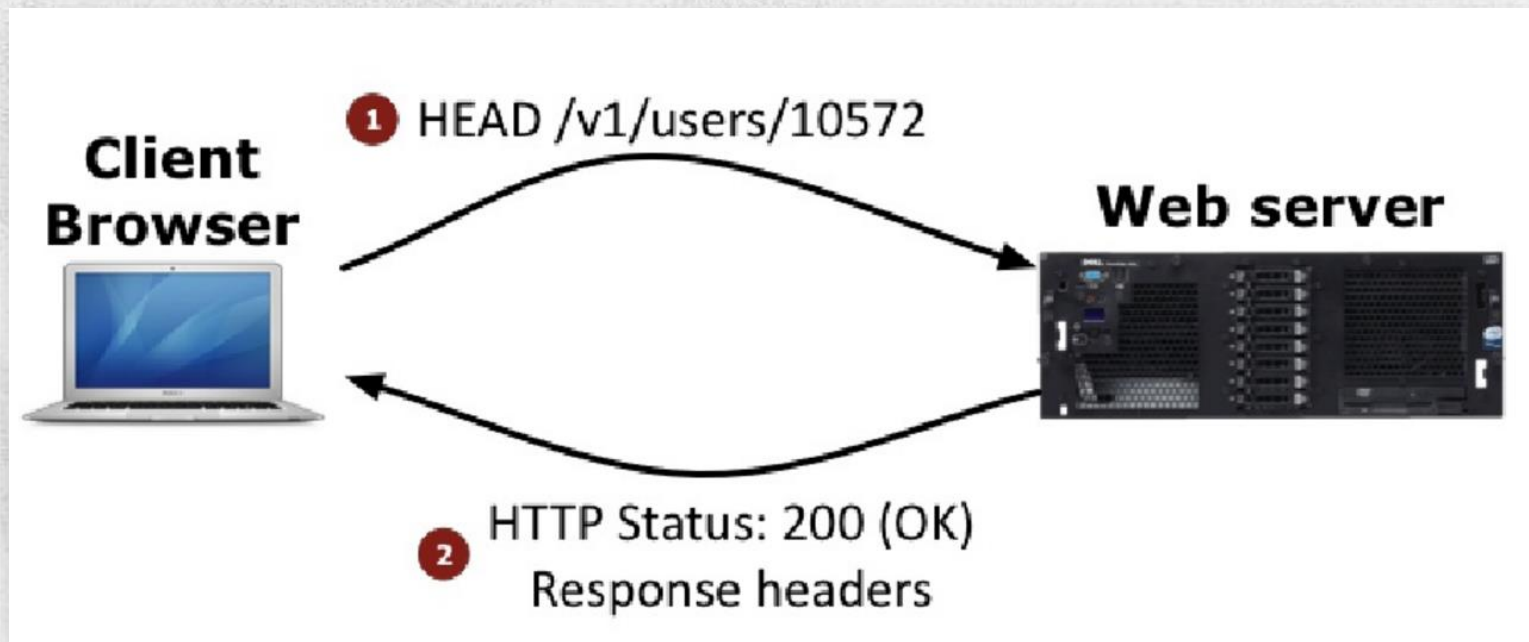
# АРХИТЕКТУРА ОРИЕНТИРАНА КЪМ РЕСУРСИ

- Стандартизирани методи
  - GET
  - PUT
  - POST
  - DELETE
  - HEAD
  - OPTIONS
- Представят действието, което трябва да се извърши
- Ресурсите имат унифициран интерфейс

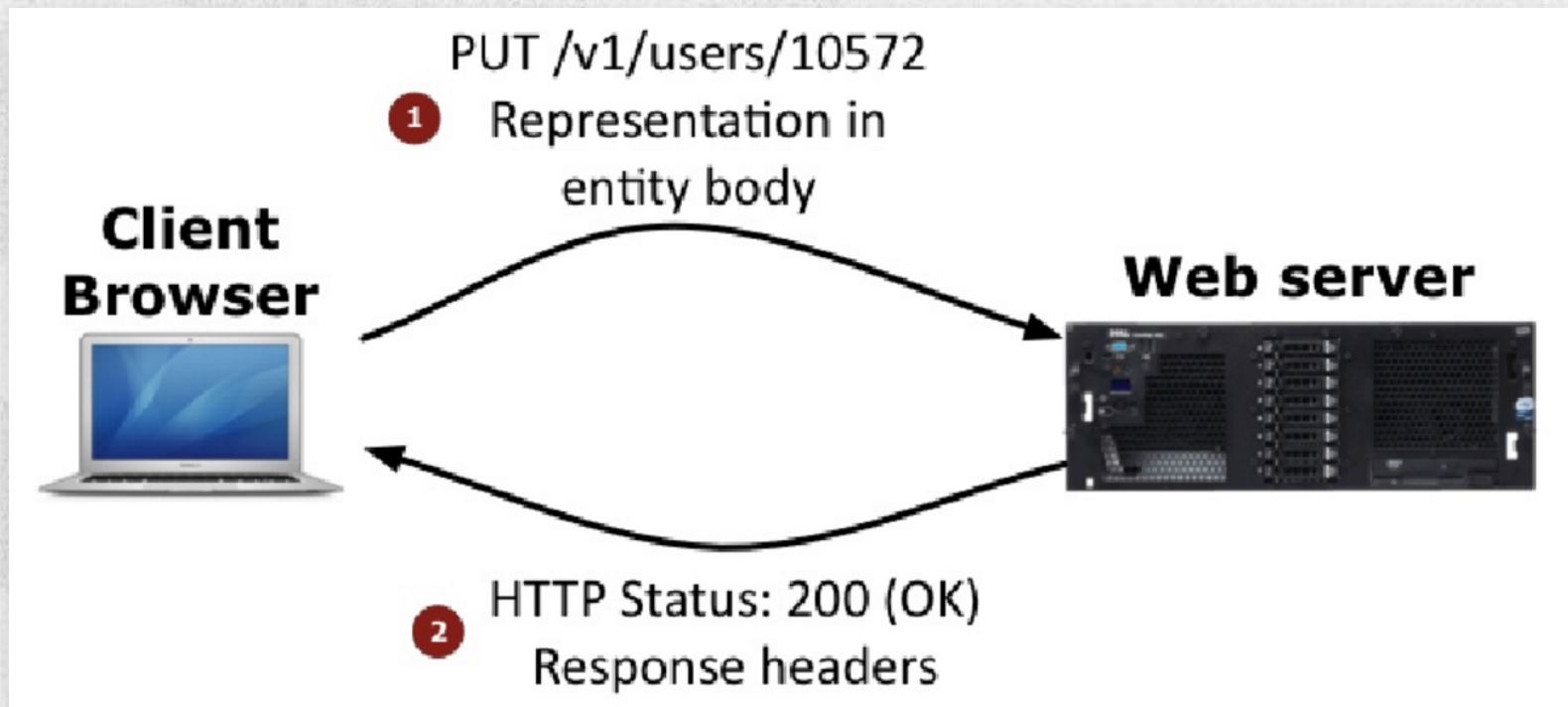
# GET



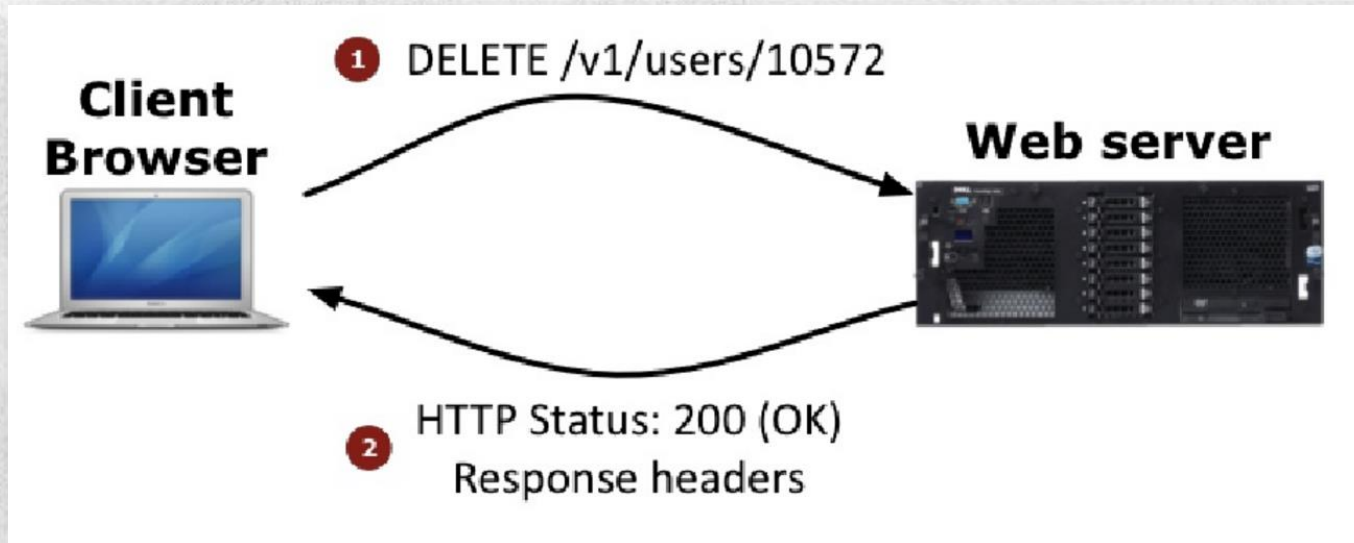
# HEAD



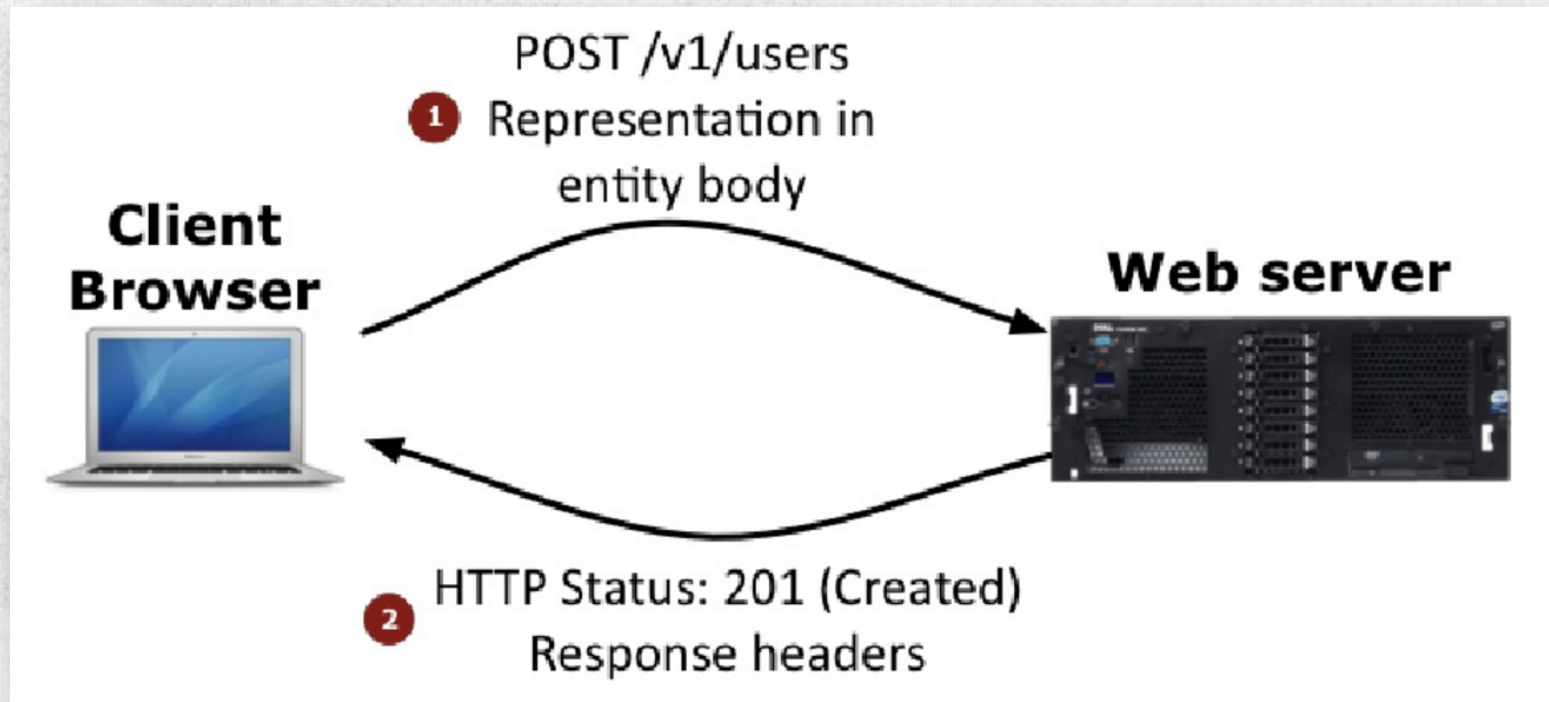
# PUT



# DELETE



# POST



# HTTP STATUS CODES

- Основен механизъм за индикация за резултата от REST операция
- Да не се връща репрезентация на ресурса за заявки различни от GET – това има отрицателен ефект при оптимизацията на производителността

# БЕЗОПАСНОСТ

- GET и HEAD операциите не трябва да причиняват катастрофални промени от страна на сървъра
- Без странични ефекти!
  - Клиентите не трябва да правят заявки само заради страничните ефекти.
- Да не се дава достъп до не безопасни операции чрез GET.

# УСТОЙЧИВИ ОПЕРАЦИИ

- Английският термин: idempotency
- Множество извиквания на такава операция водят до същия резултат както и единично извикване
- Такива методи са:
  - GET
  - HEAD
  - PUT
  - DELETE
  - OPTIONS
- Изключение: POST

# ИЗИСКВАНИЯ КЪМ REST КЛИЕНТ

- Валидиране на HTTPS и SSL сертификати
- Поддържане на 5 основни HTTP метода:
  - GET, HEAD, POST, PUT, DELETE
- Задаване на данните, които се изпращат в тялото на заявката при PUT и POST
- Задаване на HTTP хедъри
- Достъп до статуса на отговора и неговите хедъри
- Комуникация през HTTP прокси

# ДОПЪЛНИТЕЛНИ ИЗИСКВАНИЯ КЪМ REST КЛИЕНТ

- Боравене с компресирани данни:
  - Request: Accept-Encoding Header
  - Response: Encoding Header
- Кеширане на отговорите
  - Условен GET
- Поддържане на HTTP Authentication
  - Basic, Digest, WSSE
- Прозрачно следване на HTTP Redirects

# ИЗГРАЖДАНЕ НА URI

- Йерархично:
  - Данните се намират в променливи в URI
  - <http://maps.example.com/Earth/Europe/Bulgaria/Plovdiv>
- Без йерархия
  - Данните се намират в една променлива на URI, разделени с запетая или точка и запетая
  - Когато подредбата е от значение се използват запетаяи, иначе – точка и запетая
  - <http://maps.example.com/Earth/24.9295,17.821>
- Алгоритмични ресурси
  - Използват се променливи на заявката
  - <http://www.google.com/search?q=rest>

# ТИП НА СЪДЪРЖАНИЕТО

- Ресурсите могат да бъдат доставяни в различни формати:
  - XML, JSON, HTML, ...
- Методи за определяне на типа на съдържанието:
  - Headers:
    - Accept, Content-Type
  - Параметри на заявката
    - GET /books/3652?format=json
  - Разширение в URI:
    - GET /books/3652.xml

# ЧЕСТО СРЕЩАНИ ГРЕШКИ ПРИ REST

- Използване на REST като RPC механизъм
- Прекомерна употреба на POST
- Постановяне на действия в URI
- Прикриване на услуга като ресурс
- Поддържане на сесия на сървъра